

Les nouveautés de C# 6

Par [Thomas Levesque](#)  

Date de publication : 14 avril 2014

Dernière mise à jour : 13 avril 2015

TOUT PUBLIC

Mise à jour : Il y a environ un an, Microsoft dévoilait les premiers éléments sur les nouveautés de la version 6 du langage C#. Tout était encore un peu flou, la liste des features n'étant pas définitive et le design pas figé, mais maintenant qu'on approche de la sortie finale, on y voit un peu plus clair, il est donc temps de faire le point ! J'ai donc mis à jour cet article pour tenir compte des changements qui ont eu lieu depuis sa publication.

Lors de la conférence BUILD 2014, Microsoft a fait plusieurs annonces importantes ; la plus marquante, de mon point de vue de développeur, est que le projet Roslyn est maintenant open source ! Pour rappel, il s'agit du nouveau compilateur pour C# et VB.NET. Du coup, bien que le projet ne soit pas encore terminé, la liste des nouveautés des langages est déjà disponible, et on peut même commencer à jouer avec certaines d'entre elles. Cet article se propose de faire un petit tour d'horizon des nouveautés prévues pour C# 6.

Commentez

Introduction.....	3
1 - Fonctionnalités incluses dans C# 6.....	3
1-1 - Initialiseurs de propriétés automatiques et propriétés automatiques en lecture seule.....	3
1-2 - Imports statiques.....	4
1-3 - Membres dont le corps est une expression.....	4
1-4 - Initialiseurs de dictionnaires et de membres indexés.....	5
1-5 - await dans les blocs catch et finally.....	5
1-6 - Filtres d'exception.....	6
1-7 - Propagation de null.....	7
1-8 - Interpolation de chaînes.....	8
1-9 - Opérateur nameof.....	8
2 - Fonctionnalités abandonnées ou reportées.....	9
2-1 - Constructeurs primaires.....	9
2-2 - Syntaxe courte « \$ » pour les initialiseurs de dictionnaire.....	10
2-3 - Accès aux membres indexés.....	11
2-4 - Expressions de déclaration.....	11
2-5 - Littéraux binaires et séparateurs de chiffres.....	12
2-6 - Initialiseurs d'évènements.....	12
2-7 - Opérateur point-virgule.....	12
2-8 - Private protected.....	13
2-9 - Params IEnumerable.....	13
2-10 - Inférence de type pour les constructeurs.....	14
Conclusion.....	14
Remerciements.....	15

Introduction

Commencé par Microsoft il y a quelques années, le projet Roslyn est une refonte complète des compilateurs C# et VB.NET. Alors que les compilateurs sont traditionnellement des « boîtes noires », Roslyn expose ses mécanismes internes (lexer, parser, analyse sémantique, etc.) via des API manipulables depuis le code. Le projet a récemment été rendu open source et est **disponible sur GitHub**.

En plus de cette nouvelle architecture, le compilateur Roslyn implémente les fonctionnalités des nouvelles versions de C# et VB.NET. Ces fonctionnalités ont beaucoup changé au cours de l'année écoulée, suite aux échanges avec la communauté sur CodePlex puis sur GitHub. Certaines ont été abandonnées, d'autres ont été ajoutées, d'autres enfin ont un peu changé par rapport à ce qui était prévu. **Cette page** résume le statut des nouvelles fonctionnalités.

Cet article se propose de faire le tour des nouveautés de C# 6 telles qu'elles existent à ce jour. Il ne devrait en principe plus y avoir de changements d'ici à la sortie finale.

Les noms des fonctionnalités n'ont pas encore de traduction officielle, à ma connaissance. Je donne donc ma traduction, mais il est possible que le nom en français soit différent quand le langage sera finalisé.



D'autre part, je ne suivrai pas systématiquement l'ordre indiqué sur la page de statut des fonctionnalités, par exemple s'il me semble important de présenter une fonctionnalité avant une autre.

1 - Fonctionnalités incluses dans C# 6

Les fonctionnalités que je vais présenter ici sont implémentées dans la version actuelle de Roslyn ; vous pouvez les tester en installant la version CTP (Community Technology Preview) de Visual Studio 2015, téléchargeable depuis **cette page**.

1-1 - Initialiseurs de propriétés automatiques et propriétés automatiques en lecture seule

Les **propriétés implémentées automatiquement** (ou propriétés automatiques) sont apparues en C# 3, pour simplifier la déclaration de propriétés qui se contentent d'encapsuler l'accès à des champs. Bien qu'elles permettent de rendre le code plus concis, elles présentent un inconvénient : il n'est pas possible de les initialiser au niveau de la déclaration, il faut forcément le faire dans le constructeur. De plus, il n'est pas possible de faire des propriétés automatiques en lecture seule, puisqu'elles n'ont pas de mutateur (*setter*) et on ne pourrait donc pas leur affecter de valeur.

C# 6 remédie à ce problème en permettant d'initialiser les propriétés automatiques au niveau de la déclaration :

```
public int Answer { get; set; } = 42;
```

La propriété `Answer` aura donc pour valeur initiale 42.

Il est également possible d'initialiser des propriétés en lecture seule :

```
public int Answer { get; } = 42;
```

La valeur de cette propriété sera donc 42, et ne pourra pas être modifiée, sauf depuis le constructeur. Notez que dans ce cas, le champ généré par le compilateur pour stocker la valeur de la propriété est marqué `readonly`.

1-2 - Imports statiques

Le principe de cette fonctionnalité est très simple : de la même façon qu'une clause `using` sur un namespace (espace de nom) permet d'importer dans le contexte courant tous les types déclarés dans ce namespace, une clause `using static` sur une classe permet d'importer tous les membres statiques de cette classe ; ils seront donc appelables comme s'ils étaient déclarés dans la classe courante. Notez que le mot-clé `static` a été ajouté plus tard ; le design initial prévoyait d'utiliser simplement `using`, comme pour un namespace.

Un exemple avec la classe `Math` :

```
using static System.Math;
...
double x = 3;
double y = 4;
double distance = Sqrt(x * x + y * y);
```

Remarquez l'appel de la méthode `Sqrt` : jusqu'ici, on devait toujours l'appeler en précisant le nom de sa classe, en écrivant `Math.Sqrt`. Cette fonctionnalité permet d'y accéder directement, ce qui est très pratique, par exemple, dans le cas de classes qui font beaucoup d'opérations mathématiques.

Un autre bénéfice de cette fonctionnalité est la possibilité d'importer toutes les méthodes d'extension définies dans une classe donnée, ce qui permet une granularité plus fine : en effet, en important un namespace complet, on importe les méthodes d'extension de toutes les classes de ce namespace, ce qui augmente le risque de rencontrer des conflits de nom.

1-3 - Membres dont le corps est une expression

Désolé pour cette traduction bancal, le terme anglais (*expression-bodied members*) est assez difficile à rendre en français...

Le principe de cette fonctionnalité est de simplifier la déclaration de méthodes et propriétés dont le corps est constitué d'une seule instruction `return`.

Si on a par exemple une classe `Point` qui représente un point défini par ses coordonnées `X` et `Y`, on pourrait lui ajouter une propriété `Distance` qui représente la distance à l'origine, implémentée comme ceci :

```
public double Distance
{
    get
    {
        return Sqrt(X * X + Y * Y);
    }
}
```

Cela fait quand même pas mal de syntaxe inutile pour simplement retourner le résultat d'une formule... C# 6 propose une nouvelle syntaxe pour simplifier ce genre de choses :

```
public double Distance => Sqrt(X * X + Y * Y);
```

C'est déjà nettement plus concis ! Remarquez l'utilisation de la flèche (qui rappelle les expressions lambda) pour introduire le corps de la propriété. D'autre part, comme dans une expression lambda, on indique juste l'expression à renvoyer, sans le mot-clé `return`.

On peut faire la même chose avec une méthode. Par exemple, si on ajoute à notre point une méthode `Move` :

```
public Point Move(int dx, int dy) => new Point(X + dx, Y + dy);
```

1-4 - Initialiseurs de dictionnaires et de membres indexés

C# 3 avait introduit la possibilité d'initialiser des collections avec une syntaxe simple et intuitive :

```
var fruits = new List<string>
{
    "banana",
    "strawberry",
    "apple"
};
```

Pour les dictionnaires, c'était également possible, mais nettement moins intuitif, puisqu'il fallait en fait insérer des paires clé-valeur :

```
var letters = new Dictionary<string, int>
{
    { "a", 1 },
    { "b", 2 },
    { "c", 3 },
    ...
};
```

En C# 6, une nouvelle possibilité est proposée :

```
var letters = new Dictionary<string, int>
{
    ["a"] = 1,
    ["b"] = 2,
    ["c"] = 3,
    ...
};
```

Cela ressemble beaucoup plus aux initialiseurs d'objets, où on initialise chaque membre séparément ; les éléments d'un dictionnaire peuvent en effet être vus comme des membres.

Il est à noter que cette fonctionnalité n'est pas limitée aux dictionnaires, mais fonctionne avec n'importe quel type qui a un indexeur (ou plusieurs).

La possibilité d'utiliser \$a au lieu de ["a"] a finalement été abandonnée, notamment à cause du retour négatif de la communauté (voir la section §2.2 pour plus de détails).

1-5 - await dans les blocs catch et finally

Le mot-clé `await`, introduit en C# 5, permet d'interrompre l'exécution d'une méthode asynchrone en attendant la fin d'une tâche, et de reprendre plus tard là où on en était, quand la tâche attendue est terminée. Malheureusement, il y avait en C# 5 une limitation assez fâcheuse : le mot-clé `await` ne pouvait pas être utilisé dans les blocs `catch` et `finally`. Un exemple classique de cas où cette limitation est gênante : afficher un message d'erreur dans une application Windows Store. Le code qu'on souhaiterait écrire est le suivant :

```
private async Task DoSomethingAsync()
{
    try
    {
        // Some async code that can throw an exception
        ...
    }
    catch (Exception ex)
    {
        var dialog = new MessageDialog("Something went wrong!");
        await dialog.ShowAsync();
    }
}
```

```
}
```

Mais ce n'est malheureusement pas possible, puisque le mot-clé `await` est interdit dans le bloc `catch`. Il fallait donc utiliser un contournement comme celui-ci :

```
private async Task DoSomethingAsync()
{
    bool error = false;
    try
    {
        // Some async code that can throw an exception
        ...
    }
    catch (Exception ex)
    {
        error = true;
    }

    if (error)
    {
        var dialog = new MessageBox("Something went wrong!");
        await dialog.ShowDialog();
    }
}
```

Ce qui est quand même beaucoup moins pratique...

Eh bien, bonne nouvelle, cette limitation est levée en C# 6, et on peut donc maintenant écrire le code précédent comme on le souhaitait au départ.

1-6 - Filtres d'exception

Le CLR possède une fonctionnalité appelée « filtres d'exception », qui permet, avant d'entrer dans un bloc `catch`, de tester une condition. Cette fonctionnalité était déjà utilisée dans VB.NET, mais C# ne l'exploitait pas jusqu'à maintenant. C# 6 remédie à cela en ajoutant la fonctionnalité au langage. On peut maintenant écrire, pour chaque bloc `catch`, une condition qui doit être vérifiée pour que le bloc soit exécuté, ce qui permet d'être plus sélectif sur les exceptions interceptées.

Prenons par exemple le code suivant, qui n'utilise pas de filtres d'exception :

```
try
{
    using (var stream = new FileStream(path, FileMode.OpenOrCreate, FileAccess.Write))
    {
        ...
    }
}
catch(IOException ex)
{
    if (ex.HResult == 0x80070020)
    {
        // File is in use by another process
        // Handle that case...
    }
    else if (ex.HResult == 0x80070035)
    {
        // Network path was not found
        // Handle that case...
    }
    else
    {
        // Something else happened
        // Handle that case...
    }
}
```

```
catch(Exception ex)
{
    // Something else happened
    // Handle that case...
}
```

Ce n'est pas extrêmement clair, et une partie du code est dupliquée (le cas « something else happened »). Avec les filtres d'exception, on peut le réécrire comme ceci :

```
try
{
    using (var stream = new FileStream(path, FileMode.OpenOrCreate, FileAccess.Write))
    {
        ...
    }
}
catch(IOException ex) if (ex.HResult == 0x80070020)
{
    // File is in use by another process
    // Handle that case...
}
catch(IOException ex) if (ex.HResult == 0x80070035)
{
    // Network path was not found
    // Handle that case...
}
catch(Exception ex)
{
    // Something else happened
    // Handle that case...
}
```

C'est déjà plus clair ! Les différents cas d'erreur sont plus nettement séparés, et le traitement par défaut est à un seul endroit.

On pourrait croire que ça revient à tester la condition dans le `catch`, et à relancer l'exception avec `throw` si elle n'est pas vérifiée, mais c'est en fait un peu différent : en effet, une fois qu'on est rentré dans un bloc `catch`, les autres `catch` du même `try` sont ignorés ; le fait de relancer l'exception depuis un `catch` ne permet donc pas qu'elle soit interceptée par les `catch` suivants. Avec les filtres d'exception, on ne rentre pas du tout dans le `catch` si la condition n'est pas vérifiée, et on passe directement au suivant. Cela a d'ailleurs un autre avantage important : puisqu'on n'est pas rentré dans un bloc `catch`, l'état de la pile d'appels est conservé, si bien qu'en mode débogage, si l'exception n'est finalement pas gérée, le débogueur s'arrêtera à l'endroit où l'exception d'origine a été lancée, et non pas là où elle est interceptée puis relancée ; cela permet d'avoir accès à tout le contexte de la méthode où l'erreur s'est produite, et facilite donc le débogage.

1-7 - Propagation de null

Cette fonctionnalité, demandée depuis très longtemps par les développeurs C#, introduit un nouvel opérateur, `?.`, qui s'utilise de la même manière que le point pour accéder aux membres d'un objet, mais avec une différence importante : il ne lève pas de `NullReferenceException` dans le cas où l'objet est `null`. Par exemple, le code suivant :

```
string name = customer?.Name;
```

Est en fait équivalent à :

```
string name = customer != null ? customer.Name : null;
```

Autrement dit, `Name` n'est évalué que si `customer` n'est pas `null`, sinon l'expression renvoie `null`.

Il est également possible de chaîner les accès aux membres de la façon suivante :

```
string city = order?.Customer?.Address?.City;
```

Si l'un des éléments de la chaîne est null, le résultat final sera null, sans qu'on ait besoin de vérifier un par un tous ces éléments.

Dans le cas où le dernier élément de la chaîne est un type valeur, le type de l'expression sera la version *nullable* de ce type :

```
int? length = order?.Customer?.Address?.City?.Length;
```

Et il sera donc bien sûr possible d'utiliser l'opérateur `??` pour spécifier une valeur par défaut :

```
int length = order?.Customer?.Address?.City?.Length ?? 0;
```

1-8 - Interpolation de chaînes

Il s'agit là encore d'une demande très populaire des développeurs C#.

Cette fonctionnalité vise à faciliter le formatage de valeurs dans des chaînes de caractères, un peu à la façon de la méthode `String.Format`, mais avec des noms à la place des numéros.

Par exemple :

```
string s = $"Bonjour {user.Name}, nous sommes le {DateTime.Today:D}";
```

`name` et `date` étant des variables. Notez le préfixe « \$ » devant la chaîne, qui indique qu'il s'agit d'une chaîne interpolée.

Le code ci-dessus est exactement équivalent à ceci :

```
string s = string.Format("Bonjour {0}, nous sommes le {1:D}", user.Name, DateTime.Today);
```

Le code utilisant l'interpolation de chaîne est nettement plus facile à lire et à écrire.

Un point important à noter est que dans une chaîne interpolée, les valeurs sont formatées en utilisant la culture courante. Ce point a été assez vivement débattu pendant la phase de conception ; le principal argument qui a fait pencher la balance dans ce sens est le souci de cohérence avec le comportement de `String.Format`, qui utilise également la culture courante par défaut.

Heureusement, il est possible de changer ce comportement, en tirant parti du fait qu'une chaîne interpolée est implicitement convertible en un objet qui implémente `IFormattable`, qu'on peut ensuite formater dans la culture voulue. En fait, ce mécanisme est très souple et permet un contrôle du formatage beaucoup plus poussé que le simple changement de la langue. Vous trouverez des exemples dans [l'article](#) que j'ai écrit à ce sujet.

1-9 - Opérateur nameof

Cet opérateur permet de renvoyer le nom d'un symbole sous forme de chaîne de caractères :

```
string s = nameof(Console.Write); // renvoie "Write"
```

Cela est très utile pour éviter de saisir des chaînes en dur dans le code pour faire référence à un symbole : en effet, le compilateur ne vérifie pas le contenu des chaînes de caractères, et ne détectera donc pas une éventuelle faute de frappe. Avec l'opérateur `nameof`, le symbole *doit* exister, faute de quoi le compilateur produira une erreur.

Un autre avantage est que les outils de refactoring peuvent prendre en compte le symbole passé à `nameof`, et donc modifier automatiquement l'instruction utilisant `nameof` si le symbole est renommé (ce qu'on a vite fait d'oublier quand le nom est dans une chaîne de caractères).

2 - Fonctionnalités abandonnées ou reportées

Un certain nombre de fonctionnalités initialement prévues ou envisagées ont finalement été abandonnées, généralement parce que l'équipe C# a jugé que leur conception n'était pas assez aboutie pour les inclure en l'état dans le langage. Dans certains cas, elles ont été simplement reportées et seront peut-être incluses dans une version ultérieure du langage. Les sections suivantes décrivent les fonctionnalités telles qu'elles étaient prévues initialement, à titre informatif.

2-1 - Constructeurs primaires

Mise à jour : *bien que cette fonctionnalité ait été implémentée dans les premières versions publiées de Roslyn, elle a finalement été abandonnée. L'idée reste cependant envisagée pour une future version de C#, probablement sous une autre forme (types « record », voir [ce ticket sur GitHub](#) pour plus de détails).*

Cette fonctionnalité permet de faciliter la création de types qui encapsulent des données, en simplifiant la syntaxe du constructeur pour initialiser ces données. Je montrerai des exemples avec des classes, mais cette fonctionnalité s'applique également aux structures.

Une classe avec un constructeur primaire ressemble à ceci :

```
public class Point(int x, int y)
{
    public int X { get; } = x;
    public int Y { get; } = y;
}
```

Sur la ligne 1, on voit que la liste de paramètres du constructeur primaire est placée immédiatement après le nom de la classe, plutôt que dans la signature d'un constructeur explicite dans le corps de la classe.

Sur les lignes 3 et 4, on note que les propriétés sont initialisées à partir des paramètres du constructeur primaire. En effet, ces paramètres sont accessibles dans les initialiseurs des membres d'instance.

À titre de comparaison, la classe équivalente sans constructeur primaire ressemblerait à ceci :

```
public class Point
{
    private readonly int _x;
    private readonly int _y;

    public Point(int x, int y)
    {
        _x = x;
        _y = y;
    }

    public int X { get { return _x; } }
    public int Y { get { return _y; } }
}
```

On voit donc clairement l'intérêt de cette fonctionnalité : elle permet d'écrire beaucoup plus rapidement des structures de données simples. On peut espérer que cela encourage l'utilisation de structures de données immuables (non modifiables), qui sont intrinsèquement thread-safe et causent généralement beaucoup moins de bugs.

Une classe qui déclare un constructeur primaire peut également avoir d'autres constructeurs, mais ceux-ci devront alors appeler le constructeur primaire, à l'aide du mot-clé `this` :

```
class Point(int x, int y)
{
    public Point(int x, int y, string name) : this(x, y)
    {
        Name = name;
    }

    public int X { get; } = x;
    public int Y { get; } = y;

    public string Name { get; private set; }
}
```

Il est également possible d'hériter d'une autre classe en passant des paramètres au constructeur de la classe de base :

```
class Point3D(int x, int y, int z) : Point(x, y)
{
    public int Z { get; } = z;
}
```

Remarquez que les paramètres sont passés au constructeur de la classe de base directement au niveau de la déclaration de la classe de base, alors que pour les constructeurs classiques on utilise le mot-clé `base`.

Dernier point important : par défaut, les paramètres d'un constructeur primaire ne sont accessibles que dans les initialiseurs de membres d'instance ; ils ne sont pas conservés au-delà de la phase d'initialisation. Mais il est possible de spécifier qu'ils doivent être capturés dans des champs ; ils resteront alors accessibles dans n'importe quelle méthode ou propriété. Pour cela, il suffit de préciser un niveau d'accessibilité :

```
class Point(private readonly int x, private readonly int y)
{
    public int X { get { return x; } }
    public int Y { get { return y; } }
}
```

Remarquez que l'on peut spécifier, en plus de l'accessibilité, des modificateurs comme `readonly` ou `volatile`.

2-2 - Syntaxe courte « \$ » pour les initialiseurs de dictionnaire

Mise à jour : cette fonctionnalité ayant reçu un accueil plutôt négatif de la communauté, elle a été retirée.

Dans le cas où l'index (ou la clé) est une chaîne de caractères, comme ci-dessus, il existe un raccourci :

```
var letters = new Dictionary<string, int>
{
    $a = 1,
    $b = 2,
    $c = 3,
    ...
};
```

Cependant, cela ne fonctionne que si la clé est un identifiant valide. Dans le cas contraire, il faudra utiliser la forme longue, avec les crochets et les guillemets.

Un des principaux cas d'utilisation de cette fonctionnalité concerne la manipulation d'objets dynamiques, par exemple des objets JSON :

```
var joe = new JObject
{
    $id = 42,
    $name = "Joe",
    $address = new JObject
    {
        $streetAddress = "1 rue du Chateau",
        $city = "Trifouillis les Oies",
        $zipCode = "99999"
    }
};
```

2-3 - Accès aux membres indexés

Mise à jour : cette fonctionnalité a été retirée, pour la même raison que la précédente et par souci de cohérence.

Dans la continuité de la fonctionnalité précédente, C# 6 permet aussi l'accès aux membres indexés via une syntaxe simplifiée (encore une fois, uniquement pour les index de type chaîne de caractères). Si on reprend l'objet initialisé dans l'exemple précédent, on peut accéder à ses membres de la façon suivante :

```
int id = (int)joe.$id;
string city = (string)joe.$address.$city;
```

Le code équivalent sans cette fonctionnalité serait le suivant :

```
int id = (int)joe["id"];
string city = (string)joe["address"]["city"];
```

2-4 - Expressions de déclaration

Mise à jour : cette fonctionnalité posait des problèmes concernant la portée de la variable ainsi déclarée. Aucune solution intuitive et consensuelle n'ayant pu être trouvée, la fonctionnalité a été abandonnée.

Jusqu'ici, une variable ne pouvait être déclarée que dans une instruction de déclaration autonome ; il n'était pas possible de déclarer une variable en plein milieu d'une expression. Cela rendait parfois la syntaxe peu pratique, notamment dans le cas de paramètres `out` :

```
decimal price;
if (decimal.TryParse(txtPrice.Txt, out price))
    product.Price = price;
```

C# 6 change cela en permettant de déclarer la variable à l'endroit où on en a besoin :

```
if (decimal.TryParse(txtPrice.Txt, out decimal price))
    product.Price = price;
```

Un autre exemple :

```
Console.WriteLine("Result: {0}", (int x = GetValue()) * x);
```

(évite d'avoir à évaluer `GetValue` deux fois, ou de déclarer `x` séparément)

Ou encore :

```
while ((string line = reader.ReadLine()) != null)
{
    ...
}
```

(évite d'avoir à déclarer `line` avant le corps de la boucle)

2-5 - Littéraux binaires et séparateurs de chiffres

Mise à jour : ces fonctionnalités n'ont finalement pas été incluses dans C# 6. Elles ne sont cependant pas tout à fait abandonnées, il est probable qu'elles se retrouvent dans la version suivante de C# (voir ces deux tickets sur GitHub : [#215](#) , [#216](#)).

Ces deux fonctionnalités visent à faciliter l'écriture de valeurs numériques littérales dans le code.

Pour les valeurs numériques entières, C# supporte actuellement la base 10 (décimal) et la base 16 (hexadécimal). Mais dans certains cas, il serait plus pratique de pouvoir écrire des nombres directement en binaire, notamment pour spécifier des flags ou faire des opérations avec des masques de bits. Ce sera désormais possible en C# 6, en préfixant la valeur par `0b` (ce qui n'est pas sans rappeler le préfixe `0x` utilisé pour les valeurs hexadécimales) :

```
byte value = ...;
byte mask = 0b00000111;
if ((value & mask) != 0)
    ...
```

D'autre part, il n'est pas toujours facile de lire des nombres longs quand il n'y a aucun groupement de chiffres ; C# 6 permettra de regrouper les chiffres en séparant les groupes par le symbole `_` (underscore) :

```
byte binary = 0b0001_1000;
int hex = 0xAB_BA;
int dec = 1_234_567_890;
```

2-6 - Initialiseurs d'évènements

Mise à jour : cette fonctionnalité a été abandonnée, sans qu'une raison officielle n'ait été donnée à ma connaissance. On peut supposer que sa valeur ajoutée a été jugée trop faible pour justifier l'effort nécessaire à son implémentation.

Il s'agit d'une extension de la syntaxe d'initialiseur d'objet introduite en C# 3. En plus des propriétés, il est maintenant possible de s'abonner à un évènement directement dans l'initialiseur, avec l'opérateur habituel `+=` :

```
var button = new Button
{
    Content = "Say hello",
    Click += (sender, e) => MessageBox.Show("Hello world!");
};
```

J'ai utilisé ici une expression lambda pour spécifier le gestionnaire d'évènement, mais il est bien sûr possible d'utiliser un nom de méthode à la place.

Je ne m'attarde pas sur cette fonctionnalité qui, quoique bien pratique, ne soulève pas beaucoup de questions...

2-7 - Opérateur point-virgule

Mise à jour : cette fonctionnalité a été abandonnée, d'une part faute de temps, et d'autre part parce que sa conception n'était pas complètement satisfaisante. Elle pourrait éventuellement être réétudiée pour une future version.

Il s'agit ici de permettre d'écrire des expressions « composites », c'est-à-dire une série d'instructions qui retourne une valeur. Il serait ainsi possible d'écrire ce genre de choses :

```
int result = (var x = Foo(); Write(x); x * x);
```

Dans la même expression, on déclare et on initialise une variable, on appelle une méthode, et on renvoie un résultat final.

Je ne m'étendrai pas sur cette fonctionnalité, car elle est pour l'instant très peu documentée, et je risquerais donc de dire des bêtises...

2-8 - Private protected

Mise à jour : cette fonctionnalité a été abandonnée, en partie à cause de sa faible valeur ajoutée, et faute d'un consensus sur le mot-clé à utiliser.

C# 6 introduit un nouveau niveau d'accessibilité pour les membres de classe : `private protected`. Là où `protected internal` signifie « `protected` OU `internal` », ce nouveau niveau signifie « `protected` ET `internal` ».

Précisons un peu : quand un membre est déclaré `protected internal`, cela signifie qu'il est accessible depuis les classes qui :

- sont dans le même assembly
OU
- sont dérivées de la classe qui déclare le membre.

Un membre qui est déclaré `private protected` est accessible depuis les classes qui :

- sont dans le même assembly
ET
- sont dérivées de la classe qui déclare le membre.

C'est donc un peu plus restrictif que `protected internal`.

En fait, ce niveau d'accessibilité existe dans le CLR, mais n'était pas utilisable en C#. En pratique, vous aurez probablement très peu (voire jamais) l'occasion de l'utiliser ; il s'adresse principalement aux auteurs de bibliothèques de classes.

Le nom actuel est encore susceptible d'être modifié, car il est loin de faire l'unanimité.

2-9 - Params IEnumerable

Mise à jour : cette fonctionnalité n'a finalement pas été incluse dans C# 6, faute de temps. L'équipe C# semble cependant la trouver importante, il est donc probable qu'on la retrouve dans une future version du langage.

Il est possible depuis toujours en C# de déclarer une méthode qui prend un nombre d'arguments variable, avec le mot-clé `params`. La méthode manipule ces arguments sous la forme d'un tableau :

```
void PrintValues(params string[] values)
{
    foreach (string v in values) Console.WriteLine(v);
}

...

PrintValues("Hello", "world");
```

C# 6 introduit la possibilité d'utiliser le type `IEnumerable<T>` plutôt qu'un tableau :

```
void PrintValues(params IEnumerable<string> values)
```

C'est ici un choix plus judicieux, car la méthode `PrintValues` ne tire pas parti du fait que `values` soit un tableau, puisqu'elle se contente de l'énumérer. De plus, de voir passer un tableau peut être gênant pour l'appelant dans certains cas, s'il a déjà une collection ou une séquence de valeurs qu'il veut passer à la méthode.

2-10 - Inférence de type pour les constructeurs

Mise à jour : cette fonctionnalité a été abandonnée, sans qu'une raison officielle ne soit donnée à ma connaissance.

Avec la sortie de C# 3, le mécanisme d'inférence de type a été largement amélioré de façon à pouvoir éviter, dans certains cas, de devoir spécifier explicitement les paramètres de type quand on appelle une méthode générique. Par exemple, la méthode générique suivante :

```
void Foo<T>(T value) { }
```

Peut être appelée sans préciser le type de `T` :

```
Foo(42); // équivalent à Foo<int>(42)
```

Le compilateur détermine tout seul le type de `T` d'après l'argument passé à la méthode.

Malheureusement, ce mécanisme ne s'applique pas aux constructeurs ; pour instancier une classe générique, il faut forcément spécifier les paramètres de type. Par exemple pour la classe `Tuple<T1, T2>` :

```
var t = new Tuple<int, string>(42, "hello world");
```

C# 6 permet de s'affranchir de cette limitation (à condition bien sûr que la signature du constructeur permette de réaliser l'inférence de type) :

```
var t = new Tuple(42, "hello world");
```

Conclusion

Nous arrivons au terme de cette revue des nouvelles fonctionnalités de C# 6. On notera que cette version n'introduit pas de nouveautés très importantes susceptibles de changer radicalement les pratiques de développement, contrairement aux versions 2 (génériques), 3 (Linq) ou 5 (async). Il s'agit plutôt de nombreuses petites fonctionnalités qui étaient, pour certaines, souhaitées depuis longtemps, mais qui étaient trop coûteuses à implémenter dans l'ancien compilateur par rapport aux bénéfices qu'elles apportaient. Microsoft a donc profité de la remise à plat qu'est le projet Roslyn pour réduire le « backlog » des fonctionnalités à implémenter.

Un autre élément important à noter est que toutes ces nouveautés :

- ne nécessitent pas d'évolution du CLR, et devraient donc fonctionner sur la version 4 du runtime (voire la version 2 pour certaines) ;
- elles ne sont pas liées à de nouveaux types de la bibliothèque de classes de .NET.

Cela signifie qu'on pourra utiliser le compilateur C# 6 pour cibler des versions plus anciennes de .NET. Cela représente un avantage important, notamment pour les entreprises qui doivent maintenir des applications qui fonctionnent encore sur d'anciennes versions.

Enfin, le fait que Microsoft ait décidé de rendre open source son nouveau compilateur est une petite révolution en soi. C'est en quelque sorte le couronnement du processus de réconciliation entre Microsoft et l'open source, entamé il y a quelques années avec des projets comme ASP.NET MVC. Le fait d'avoir une implémentation open source de référence du compilateur va certainement encourager l'utilisation de C# sur des plateformes autres que Windows, et donner plus de transparence sur les évolutions futures du langage.

Remerciements

Je tiens à remercier **Pongten** pour sa relecture technique, ainsi que **f-leb** pour la correction orthographique.